

HADOOP

1. What are the differences between regular FileSystem and HDFS?

Ans:

Features	Regular File System	HDFS
Storage	Usually one computer/server	Multiple machines (cluster)
Purpose	General-purpose file storage	Big Data storage
Data Size	Small to large, but limited by one machine	Designed for TBs/PBs of data
Data Distribution	Files stay on one machine	Files are split into blocks and distributed
Fault Tolerance	Usually depends on backups/RAID	Built-in through replication
Scalability	Limited by hardware	Can add more machines
Performance	Good for normal file operations	Optimized for large sequential reads/writes
Access	Local applications access files directly	Applications access data over the cluster
Typical Use	Windows/Linux/macOS files	Hadoop/Spark Big Data workloads

2. What is HDFS fault-tolerant?

Ans: HDFS automatically keeps multiple copies of data (replicas), so losing a DataNode doesn't necessarily mean losing the data.

3. What are the two types of metadata that a NameNode server holds

Ans: The two types of metadata that a NameNode server holds are:

- File system namespace: Information about the file and directories. Includes file names, directory structure, permission, ownership, and file-to-block mapping. Example:
(/data/sales.csv = Block 1, Block 2, Block 3)
- Block Locations: Information about which DataNodes store each block of a file. Example:
(Block 1 = DataNode 1, DataNode 3 Block 2 = DataNode 2, DataNode 4)

3. If you have an input file of 350 MB, how many blocks would HDFS create and what would be the size of each block? Assuming default block size is 128MB.

Ans: Input file = 350 MB

Number of blocks:

$$350 / 128 = 2.73$$

Since HDFS cannot have a fraction of a block, we round up 3 blocks:

Block sizes:

Block 1: 128 MB

Block 2: 128 MB

Block 3: 94 MB

$$128 + 128 + 94 = 350 \text{ MB}$$

HDFS will create 3 blocks: 128 MB, 128MB, and 94 MB.

4. How does rack awareness work in HDFS?

Ans: Rack awareness in HDFS means the NameNode keeps track of the physical rack location of DataNodes and uses this information when placing block replicas. Replicas are distributed across different racks to protect against rack-level failures while also minimizing network traffic.

How it works:

Original block



DataNode 1 (Rack 1)



Replica → DataNode 2 (Rack 2)



Replica → DataNode 3 (Rack 2)

5. Is it possible to change the number of mappers to be created in a MapReduce job?

Ans: The number of mappers is usually determined by the number of input splits, not directly by the number of HDFS blocks. For example, with a 350 MB file and a 128 MB block/split size:

350 MB



128 MB + 128 MB + 94 MB



3 input splits



3 mappers

6. What is the distributed cache in MapReduce?

Ans: Distributed Cache is a MapReduce feature used to share files or small read-only data with all the Mapper and Reducer nodes in a Hadoop cluster.

Q 1. **Please write down the ubuntu / hadoop commands for followings**

- Create a csv file with **bootcamp.csv** on your local ubuntu machine (docker container).
→ touch bootcamp.csv
- Copy the below data into bootcamp.csv
→ nano bootcamp.csv
→ then ctrl+c and ctrl+v into the nano editor

■

■ Upload

```
Duration,Pulse,Maxpulse,Calories
60,110,130,409.1
60,117,145,479.0
60,103,135,340.0
45,109,175,282.4
45,117,148,406.0
60,102,127,300.0
60,110,136,374.0
45,104,134,253.3
30,109,133,195.1
60,98,124,269.0
60,103,147,329.3
60,100,120,250.7
60,106,128,345.3
60,104,132,379.3
60,98,123,275.0
60,98,120,215.2
60,100,120,300.0
```

bootcamp.csv into a hadoop dir with name `/user/test/bootcamp/data`

—> Hadoop fs -put bootcamp.csv /user/test/bootcamp/data

■ Print the first 5 records from the top

—> cat bootcamp.csv | head -n 5

—> hadoop fs -cat /user/test/bootcamp.csv | head -n 5

■ Print the last 5 records from the boot

—> cat bootcamp.csv | tail -n 5

—> hadoop fs -cat /user/test/bootcamp.csv | tail -n 5

○ **Q 2. Please write down the ubuntu / hadoop commands for followings**

■ Create a dir on your local machine with the name as `data1` .

—> mkdir data1

■ Change your directory to `data1`

—> cd data1

■ Download the bootcamp.csv from hadoop `/user/test/bootcamp/data` into the local dir.

—> hadoop fs -copyToLocal /user/test/bootcamp/data bootcamp.csv data1